摘要：

董事会12日宣布，黄仁勋已辞去总裁、首席执行官及董事会成员职务，即时生效。英伟达现任执行副总裁兼首席财务官Colette Kress已同意担任临时首席执行官。董事会正启动正式首席执行官的遴选，并将考虑公司内外的候选人。董事会同时任命Harvey C. Jones为有史以来首任董事会主席。

董事会12日宣布，黄仁勋已辞去总裁、首席执行官及董事会成员职务，即时生效。

英伟达现任执行副总裁兼首席财务官Colette Kress已同意担任临时首席执行官。在正式继任者被任命之前，Kress在履行临时首席执行官之职的同时将继续兼任执行副总裁和首席财务官。**董事会正启动正式首席执行官的遴选，并将考虑公司内外的候选人。**

Kress自2013年9月加入公司以来一直担任英伟达执行副总裁兼首席财务官。在任期间，她领导了公司的财务部门并在英伟达的发展与进化上发挥了关键作用。加入英伟达之前，Kress曾在思科和微软等领先科技公司的财务岗位服务25年。

董事会同时任命Harvey C. Jones为有史以来首任董事会主席。Jones先生30多年来当过创业者、科技管理者和积极风险投资者，包括在新思科技担任首席执行官及执行主席。

“Colette协助带领英伟达度过了非凡发展的时期且对我们的业务、客户、运营以及财务状况都有着深刻的理解，” 英伟达董事会主席Harvey Jones说，“为她提供支持的还有一支经验丰富的领导团队，而后者在执行英伟达的战略以及推进技术路线图上发挥了重要作用。董事会对Colette及领导团队充满信心，我们也预计公司的运营、客户承诺以及产品路线图都将不受阻断地继续。”

“英伟达的实力始终源于其中的人。与我们的领导团队一道，我将致力于在黄仁勋过去33年所奠定的卓越基础之上更进一步，” 临时CEO兼CFO Colette Kress说，“我们将继续推动使英伟达在加速计算和人工智能领域成为领导者的各项工作，并抓住未来的发展机遇。我们始终聚焦服务我们的客户、合作伙伴、开发者以及持份者。”

黄仁勋对英伟达的贡献是深远的。英伟达是全球最具影响力的科技公司之一。董事会及公司感谢黄仁勋为公司及行业所作出的不可估量的贡献。

英伟达将按照此前的计划，在太平洋时间11月17日周二下午1:20左右公布2027财年第三季度财务报告，随后于2时举行业绩电话会。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次 VIP会员·7天畅读卡 | 大师课·入门串讲 扫码 免费领取

限免

785 收藏 分享到:

写评论

请发表您的评论

表情 图片 发布评论

6 条评论



别叫哥
1小时前 中国浙江省

没了黄仁勋的英伟达就像没了黄仁勋的英伟达

0 回复



Mark
3小时前 中国上海

自毁长城

5 回复



手机用户
3小时前 中国山东省

没了黄仁勋的英伟达还能走多远?

5 回复



MobileUser9003 VIP会员
3小时前 中国广东省

跑路啦跑路啦

0 回复



MobileUser7626
4小时前 中国广东省

就这样了?

0 回复

摘要：

克利夫兰联储行长贝丝·哈马克表示，通胀改善进程已实质性停滞，正在"朝错误方向移动"，“需要转向限制性的政策立场”。其主张以"更频繁、更小幅度"的方式稳步推进加息，而非采取激进的大幅行动。哈马克是上月FOMC会议两位投票加息的官员之一。

美联储克利夫兰联储行长Beth Hammack（贝丝·哈马克）再度呼吁采取更具限制性的利率立场，认为当前通胀形势仍未得到有效控制，人工智能投资热潮带来的需求压力进一步加大了政策收紧的迫切性。

Hammack周四在俄亥俄州哥伦布市的一场活动上表示，**"我的判断是，我们需要转向限制性的政策立场，以推动通胀回落至2%的目标——这一目标至关重要。"**

她警告称，若政策行动迟缓，可能引发通胀预期的自我强化。最新数据显示，10月份核心通胀高于预期，**她形容通胀改善进程已"实质性停滞"，并直言通胀正在"朝错误方向移动"。**

在利率路径上，**Hammack主张以"更频繁、更小幅度"的方式稳步推进加息，而非采取激进的大幅行动。**

她在上月美联储决策会议上是两位持异议票官员之一，反对将利率维持在3.75%至4%区间不变，倾向于支持再加息25个基点。

通胀数据令人失望，政策收紧理由充分

Hammack指出，近期通胀数据走势令人担忧。

10月核心通胀（剔除食品和能源）升幅超出预期，表明通胀黏性依然较强。"通胀比我希望的更为持久，"她说，"最新数据显示，通胀正在偏离2%目标，而非向其靠拢。"

她还提及人工智能基础设施建设带来的需求端压力。**随着数据中心大规模扩张，相关投资对经济活动形成明显拉动，美联储在研判通胀走势时需充分考量这一结构性因素。**

"当我审视这一平衡时，我看到的是黏性通胀，"Hammack说。

倾向渐进加息，不排除更激进选项

在加息节奏问题上，Hammack与部分同僚存在分歧。

她明确支持采取小幅、频繁的方式调整政策利率，认为这样对企业和家庭而言更具可预期性，对整体经济的扰动也更小。**"稳步推进小幅调整，比大开大合的激进动作破坏性要小得多，"她说。**

然而，官员内部对加息步伐的看法并不统一。据悉，美联储理事Christopher Waller本周警告称，美联储或需考虑在12月会议上加息50个基点。

对此，Hammack表示，若政策陷入被动落后局面，不排除采取更激进行动的可能，但她目前不认为已出现这种情形。**"我不认为我们今天已经落后于曲线，但如果真的落后了，那或许是需要考虑的选项，"**她说，并补充道，进入12月会议时她将保持开放心态。

上月会议持异议，12月立场受关注

Hammack在上月美联储利率决议中投下反对票，成为少数派之一。

彼时美联储以多数票决定将联邦基金利率目标区间维持在3.75%至4%，而Hammack则明确倾向于支持再加息25个基点。

此次表态进一步明确了她在12月会议前的政策立场。随着美联储内部在加息节奏与幅度上的讨论持续升温，12月议息决策的不确定性有所上升，市场对官员表态的关注度也随之提高。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



限免

231 收藏

分享到:

写评论

请发表您的评论

表情 图片

发布评论

2 条评论

巴扎黑
1小时前 中国海南省

美联储主席沃什是懂王提拔的，可是美联储仍然决定加息，而且懂王并没有明显的反对。只有一种情况符合这种情形，那就是美国的通胀远比官方通报的数据还要严重。另外从中国的出口顺差大幅上涨也证明欧美通胀非常严重。

0 回复

Nirvana-B
5小时前 中国

好嘛，你也来添乱

0 回复

摘要：

上周五和本周四的报道均称，OpenAI以1.25万亿美元的估值进行新一轮融资。9月末的报道称，OpenAI计划以约1.4万亿美元估值融资至少300亿美元。

OpenAI新一轮融资规模可能远不及最初计划，最新消息进一步印证了这一趋势。

据美东时间11月12日周四的CNBC报道，OpenAI新一轮融资已累计募集150亿美元，公司估值为1.25万亿美元。这一金额虽然高于上周彭博社披露的首批110亿美元，但距离OpenAI最初寻求的300亿美元或更高融资规模仍有明显差距。按照最新进展，本轮融资即使继续推进，最终规模也可能显著低于最初目标。

事实上，OpenAI的融资计划在过去一个多月已经出现明显变化。彭博社9月29日曾报道称，OpenAI最初计划以约1.4万亿美元估值融资至少300亿美元；到了11月6日上周五，一些媒体的报道称，公司仅完成首批110亿美元，整轮融资规模可能达到200亿美元。如今CNBC披露的累计融资金额升至150亿美元，但距离此前预期的200亿美元乃至最初的300亿美元目标仍有距离。

融资目标从300亿美元缩水，估值也从1.4万亿美元下调

本轮融资规模的变化，反映出AI行业融资环境正在发生转变。

彭博社9月29日的报道显示，OpenAI最初寻求以约1.4万亿美元估值融资300亿美元或更多。随后，融资估值调整至1.25万亿美元，融资规模预期也有所收缩。

11月6日，彭博社报道称，OpenAI已经完成首批约110亿美元融资，整轮融资原本可能达到200亿美元。路透社当天也援引彭博社报道了这一进展。

CNBC最新披露的150亿美元，则意味着本轮融资累计规模较上周增加约40亿美元，但仍明显低于最初设定的目标。需要注意的是，150亿美元是目前披露的累计融资金额，并不意味着融资已经正式结束，最终规模仍有待确认。

尽管融资规模缩水，OpenAI的估值依然高达1.25万亿美元。这一数字虽然低于最初计划融资时的1.4万亿美元，但仍明显高于公司今年3月完成1220亿美元融资时的8520亿美元估值。

三季度营收突破100亿美元，环比增长近50%

融资规模出现收缩之际，OpenAI的业务收入仍在快速增长。

据11月10日周二的报道，OpenAI今年第三季度营收达到100亿美元，较第二季度增长近50%，同比接近三倍。

据报道，知情人士透露，这些数据是在周二与投资者举行的电话会议上披露的。OpenAI总裁Greg Brockman、首席财务官Sarah Friar等高管参与了会议。

营收增长表明，OpenAI在企业市场和AI产品商业化方面仍保持强劲势头。不过，收入快速增长并不意味着盈利压力已经消失。AI模型研发、数据中心建设和算力采购仍需要巨额投入，如何将用户增长和企业需求转化为可持续利润，仍是投资者关注的核心问题。

Anthropic IPO在即，估值或超过OpenAI

OpenAI融资之际，主要竞争对手Anthropic也即将登陆公开市场。

按照Anthropic最新IPO发行价区间上限计算，公司完全稀释后估值约为1.4万亿美元，超过OpenAI本轮1.25万亿美元的融资估值。

这意味着，Anthropic有望在上市时获得高于OpenAI最新融资估值的市场定价。不过，两家公司的估值口径和交易性质不同，不能简单视为同一标准下的直接比较。

随着Anthropic推进IPO、OpenAI继续寻求外部融资，资本市场对AI巨头的增长潜力、商业化能力以及巨额算力投入回报的评估，也将迎来新的检验。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次
VIP会员·7天畅读卡 | 大师课·入门串讲
扫码 免费领取

限免

122 收藏 分享到:

写评论

请发表您的评论

表情 图片 发布评论

1 条评论

八闽望族之冠
4小时前 中国上海

华尔街不愿认错！继续飞蛾扑火！

0 回复

摘要：
特朗普称，那样的日子早已过去。他声称，美国已经掌握了伊朗数千亿美元的资金以及霍尔木兹海峡。

特朗普表示美国无需与伊朗达成协议，令中东局势前景更趋不明朗，市场对该地区供应风险的担忧重燃。

据央视新闻，当地时间11月12日，**美国总统特朗普表示，美国不需要与伊朗达成协议。 特朗普称，那样的日子早已过去。他声称，美国已经掌握了伊朗数千亿美元的资金以及霍尔木兹海峡。**

话音一落，原油直线拉升，布油转涨，截至发稿，日内涨幅超过1%。



特朗普表示，副总统JD·万斯应把重点放在国内事务上，尤其是调查美国各地由民主党掌管的城市和州所涉腐败问题。而据媒体此前报道，万斯正考虑与伊朗强硬派安全官员Mohsen Rezaei进行直接会谈，特朗普已向幕僚和外国对应方表示，支持万斯重启对伊谈判并授权其主导这一进程。

与此同时，伊朗方面也摆出强硬姿态。

据央视新闻12日报道，伊朗最高国家安全委员会秘书雷扎伊（Mohsen Rezaei）在11月11日播出的采访中表示，如果美国再次对伊朗发动军事打击，伊朗将把打击范围扩大至欧洲境内的美军基地。他明确警告：**"如果美国军机从欧洲国家的美军基地起飞轰炸伊朗，我们也将打击那些基地。"雷扎伊同时否认伊朗正与美国进行任何谈判，称相关报道是"谎言"，并强调在伊方七项条件得到满足之前，霍尔木兹海峡将继续关闭。**

据新华社周三报道，雷扎伊表示，**伊朗不会让一点浓缩铀运出境。**伊方已告知俄罗斯，浓缩铀将留在伊朗境内供自己使用。伊朗需要用浓缩铀设计以核能为动力的船舶和潜艇。

美国军事支持是中东油流的关键变量

特朗普言论公布后，瑞银集团分析师Giovanni Staunono指出，中东地区油流的持续畅通，在很大程度上依赖美国的军事支持，包括为油轮提供海军护航。

"这意味着供应能否持续，部分取决于美国愿意在多大程度上、多长时间内维持这一高代价的服务，"他表示。

分析人士的上述判断，点明了当前油市的核心不确定性所在：一旦美国调整在波斯湾的军事存在，中东原油出口的稳定性将面临直接冲击，投资者对地缘风险溢价的重新定价或在所难免。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次
VIP会员·7天畅读卡 | 大师课·入门串讲
扫码 免费领取



限免

500 收藏 分享到:

写评论

请发表您的评论

表情 图片 发布评论

7 条评论

FinancialDinosaur
1小时前 中国江苏省

伊朗的要求 ，共和党承担不起，只能拖延时间，伊朗需要主动制造问题，等27年窗口期之后，民主党议员会助攻的

0 回复

小虎
3小时前 中国上海

伊朗挟持海峡真的不地道

1 回复

从儿格
5小时前 中国广东省

求着签的是伊朗 川子是对的

3 回复

MobileUser9201 4小时前 中国安徽省

回复 **从儿格:**

还在这赢呢

0 回复



见闻用户 4小时前 中国上海

回复 **MobileUser9201：**

穷到发不出工资的又不是金毛

0 回复



普洱老于

5小时前 中国吉林省

只有真正动用了伊朗的被冻结资金用于赔偿受到伊朗及伊朗支持的武装组织实施的袭击造成的损失，伊朗才会能老老实实地做在谈判桌前

6 回复



MobileUser4617

5小时前 中国福建省

又开始画K乐

3 回复

摘要：

伊朗最高国家安全委员会秘书雷扎伊称，"如果美国军机从欧洲国家的美军基地起飞轰炸伊朗，我们也将打击那些基地。"他同时否认伊朗正与美国进行任何谈判，称相关报道是"谎言"，并强调在伊方七项条件得到满足之前，霍尔木兹海峡将继续关闭。

伊朗最高安全官员发出迄今最强硬的军事警告，将潜在打击目标范围扩大至欧洲境内的美军基地，令中东局势紧张态势进一步向外蔓延，并对全球能源市场构成新的不确定性压力。

据央视新闻12日报道，伊朗最高国家安全委员会秘书雷扎伊（Mohsen Rezaei）在11月11日播出的采访中表示，如果美国再次对伊朗发动军事打击，伊朗将把打击范围扩大至欧洲境内的美军基地。他明确警告：**"如果美国军机从欧洲国家的美军基地起飞轰炸伊朗，我们也将打击那些基地。"雷扎伊同时否认伊朗正与美国进行任何谈判，称相关报道是"谎言"，并强调在伊方七项条件得到满足之前，霍尔木兹海峡将继续关闭。**

上述表态令市场对中东局势的风险溢价预期升温。霍尔木兹海峡是全球重要的石油运输通道，伊朗持续关闭该海峡已对国际油价形成支撑。雷扎伊此番将打击目标延伸至欧洲，进一步加剧了市场对冲突扩大化的担忧，也使美伊之间本已脆弱的外交斡旋前景更趋复杂。

周四油价延续回调趋势，布油逼近90美元关口。



打击范围升级，欧洲美军基地被列入目标

雷扎伊在采访中系统阐述了伊朗潜在的军事反应层级：首先打击附近所有美军基地，其次打击此前尚未攻击过的远程美军基地，最后甚至可能打击"美方认为伊朗导弹无法触及的岛屿"。

他表示，**伊朗的打击能力"远超战争第一天"，并称伊朗导弹射程已覆盖此前被认为超出其能力范围的目标。**伊朗媒体此前列出的潜在打击目标包括位于塞浦路斯、希腊、罗马尼亚、阿尔巴尼亚、意大利和英国的美军及北约军事设施，以及美英在印度洋迪戈加西亚岛共同运营的军事基地。

不过，伊朗是否具备实际打击上述目标的能力，目前仍存争议。美国战略与国际问题研究中心（CSIS）的资料显示，**伊朗主要弹道导弹射程约为2000公里，部分型号可能达到3000公里。**英国皇家联合军种国防研究所则表示，**从理论上讲，伊朗具备打击英国境内目标的可能性。**

值得注意的是，**在上述警告发布前数日，美国军方已将部分空中加油机和军机从英国东部的米尔登霍尔皇家空军基地撤离，约10架KC-135加油机和3架"鱼鹰"倾转旋翼机被转移至德国、西班牙及英国其他基地，此举被认为与潜在安全威胁有关。**

强硬表态背后：打击范围的逐步扩大

雷扎伊在采访中梳理了伊朗军事行动的演变轨迹，以此为此次警告提供背景支撑。他表示，在2025年6月持续12天的战争中，伊朗将打击范围限制在以色列境内；而在2月28日爆发的本轮战争中，伊朗已对中东地区多个美国军事基地发动袭击；如今，伊朗官员开始公开将欧洲列入潜在打击范围。

上个月，伊朗最高领袖穆杰塔巴·哈梅内伊曾下令武装部队做好"即时、有力、不受地理范围限制"的报复准备，并要求全面更新目标数据库，将所有美国和以色列资产纳入打击范围。伊朗军方发言人随后声明称，下一场战争可能蔓延至"西亚以外地区"。

否认谈判，外交前景蒙上阴影

雷扎伊的强硬表态与近期有关美伊外交接触升温的报道形成直接矛盾。**他明确否认伊朗正与美国进行任何谈判，并将有关双方计划在卡塔尔举行会晤、或他本人将与美国副总统万斯会面的报道斥为"谎言"，称这些消息旨在"缓解经济压力并压低国际油价"。当被直接问及是否计划与万斯会面时，他回答："绝对没有。"**

然而，据媒体报道，**万斯近期在对伊外交中扮演的角色已超过特使Steve Witkoff和Jared Kushner。**万斯于10月向雷扎伊发出信函，请求安排会面，多哈被视为下一阶段谈判的可能举办地。**万斯上周还会见了卡塔尔首相，特朗普称此次会面"非常出色"。**

据央视新闻12日报道，伊朗议会议长卡利巴夫表示，伊朗已向美方给出了明确且直白的答复，阐述了伊朗的正当诉求。

核问题与霍尔木兹：伊朗坚守红线

在核问题上，雷扎伊态度同样强硬。

据新华社周三报道，雷扎伊表示，**伊朗不会让一点浓缩铀运出境。**伊方已告知俄罗斯，浓缩铀将留在伊朗境内供自己使用。伊朗需要用浓缩铀设计以核能为动力的船舶和潜艇。

在霍尔木兹海峡问题上，雷扎伊重申，美国必须全面满足伊方提出的七项条件，海峡方可重开。

他表示，伊方首要条件是结束所有战线上的战争，尤其是黎巴嫩战事，并称美国最初的回应具有"欺骗性"。雷扎伊还声称，在此前一轮于伊斯兰堡举行的谈判中，伊美双方曾签署一项协议，但特朗普在不到一个月后便撕毁了协议。

霍尔木兹海峡持续关闭对全球原油供应构成直接威胁。雷扎伊此番将打击目标延伸至欧洲、同时否认谈判进展的表态，令市场对局势缓和的预期再度降温，油价走势将继续受到美伊关系动向的密切牵引。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领
* 体验资格每人限申领一次
VIP会员·7天畅读卡 | 大师课·入门串讲
扫码
免费领取



限免

141 收藏

分享到:

写评论

请发表您的评论

表情 图片

发布评论

摘要：

一家伟大的公司，创始人离开是早晚的事，英特尔、苹果、微软、台积电都是在创始人离开之后走到了更高的地方，老黄只是用了一种没人想到的方式离开。没有老黄的英伟达不会有什么大的影响，只是大家不习惯而已。

这几天科技圈、资本市场都在吃黄仁勋的瓜，细节不给大家复述了。

忽略掉最终结果，我们可以探索一个问题：假如黄仁勋从CEO的位置上被下架，英伟达会怎么样，这应该也是外界更关心的一个议题。

事实上，出轨花边这种事看都不要看，核心就两个问题：是否涉及违规（美国出口管制）、是否违反公司治理的规则。

关于出口管制的问题我们不去讨论了。关于公司治理规则，CEO跟直属下属有关系，在美国公司是明确违反行为准则的事项，这涉及权力不对等跟利益冲突。2018年英特尔的CEO科再奇，2019年麦当劳的CEO伊斯特布鲁克，都因为这个问题被董事会“请”走的，不涉及违法。

按英伟达的公司章程，11位董事过半数就可以罢免CEO。所以，董事会要换人，不需要等到有明确的法律问题。这也是为什么第一个瓜出来的时候，董事会表态有信心，而到了第二个瓜涉及内部直属下属，就传出董事会讨论罢免的事项。

如果黄仁勋违反公司规则，董事会再保他，董事们自己就要面对股东诉讼。最终，是否更换老黄，具体结果还是由董事会决定。

那么回到我们的终极问题：英伟达是不是离不开老黄？没有老黄的英伟达还有是那个英伟达吗？

01 因果关系要搞明白

没有老黄，英伟达还是英伟达。

的确，好的领导者确实能更好地引领方向，是一家公司最重要的人，但产业跟技术的方向与发展，从来不是某个人之功。这里的因果关系要搞明白。

老黄是工程师出身的创始人，英伟达能有今天，靠的是三十多年对GPU这个方向的坚持，而不是老黄个人发明了多么牛的技术。而且，老黄也不是十几年前就看到了今天的AI才坚持GPU的。

GPU最早是给游戏用的，2006年推出CUDA是为了让GPU能做通用计算跟科学计算，2012年AlexNet用英伟达的游戏显卡跑出了深度学习，之后又经历了挖矿，最后才等到大模型。

每个时代有每个时代的需求，英伟达是跟着当下的需求一步一步走过来的。

英伟达的高管体系中，首席科学家Bill Dally管研究，从Mellanox过来的Michael Kagan现在是CTO，Ian Buck是CUDA的缔造者，现在管数据中心业务，下面还有四万多名员工。

如今，整个AI的方向已经明确，英伟达的产品路线也已排到了几年以后，这些东西都不是黄仁勋一个人拍板出来的，自然也不会因为老黄离开就消失。

没有老黄的坚持就没有英伟达的今天，这一点是肯定的。但是一家已经完全上轨道的公司，离开创始人还能不能走，历史早就给过答案。

苹果、英特尔、微软、亚马逊、台积电，案例多到数不清。其中，笔者的老东家台积电也值得关注。

张忠谋2005年把CEO位置交给蔡力行，2009年金融危机后张忠谋自己回任，2013年再交给刘德音跟魏哲家，2018年正式退休。如今魏哲家带领的台积电，比张忠谋时代强大了好几倍。台积电的例子还说明另一件事，交棒不一定一次就成功，但只要治理机制在，换错了人可以再换。

所以我们要清楚西方科技公司的治理模式——职业经理人主导，加上独立董事制衡。

苹果、微软、谷歌、亚马逊现在全是经理人在管，创始人还在一线主导的巨头除了英伟达，基本剩下特斯拉、Meta少数几家，这几家公司都是创始人非常鲜明甚至作为招牌。

老黄在英伟达的持股只有3.58%，也没有超级投票权，英伟达本质上早就是经理人制度的公司，只是这个经理人刚好是创始人，而且干得特别好，所以能一直干下去。

干得好就继续干，出了问题董事会换人，在西方企业巨头当中太稀松平常了。

02 管理层，谁来接

讲管理层之前，得先说英伟达一个很特别的地方——没有二把手。

英伟达不设COO，管理架构极度扁平，直接向老黄汇报的高管一度有55人，后来缩到36人，伯恩斯坦的说法更是有约60人。老黄自己也一直不愿意做正式的接班计划，他的理由是接班安排只会制造高管之间的竞争跟分裂。

英伟达这几年一路高歌猛进的时候，市场可是把这套扁平式管理奉为公司管理的典范。

客观地说，这套模式有利有弊。有利的地方是决策快，信息不用一层一层往上传，老黄能直接听到一线的声音，组织也精简，没有大公司常见的层层汇报跟内部政治。英伟达能做到一年一代产品，跟这套模式有直接关系。

弊端也很清楚：一把手一旦出问题，公司马上面临没有人能顶上的风险；另一个弊端是这套模式非常“吃人”——一个CEO要面对几十位高管听取汇报，这是极其庞大的工作量，换一个职业经理人来未必做得到。

所以这套模式是跟老黄这个人绑在一起的。老黄在的时候它是优势，老黄不在了它就需要调整，这是换人的自然结果，不代表原来的模式是错的。

市场上流传的各种接班人名单。临时CEO这个位置，笔者认为最确定的人选是CFO Colette Kress。

首先这个位置是临时CEO，只需要渡过，老黄临时卸任的这段不稳定期，不需要对行业或技术有非常深刻的建地。

Colette Kress 2013年加入英伟达，当了十三年CFO，华尔街跟大客户对她最熟，她是财务出身，不懂技术，所以最适合过渡。

内部的技术派是Ian Buck，也就是鼎鼎大名的CUDA之父，现在管超大规模跟高性能计算业务，跟几家云大厂的关系最深。他的短板是没有当过一把手，没有管理整家公司的经验。

除了Colette Kress跟Ian Buck这两个热门人选，英伟达内部的CTO跟几个副总裁也是内部可能人选，内部依旧是人才济济。

外部人选方面，Semafor点名的是苹果的硬件芯片负责人Johny Srouji，投行Baird也表态倾向外部空降。至于市场上传的AMD苏姿丰、Arm的Rene Haas，目前都只是媒体跟分析师的猜测，没有任何实锤。

笔者的判断是，最可能的组合是Kress临时过渡，同时由外部猎头找正式CEO，Ian Buck这些技术高管负责稳住产品跟客户。

如此一来，新的管理层大概率会设COO，把几十条汇报线收成常规的层级。这样决策速度会慢一些，但公司不再依赖某一个人，这是进入经理人时代必然的取舍。

03 三种可能



笔者把这件事的走向归纳成三种情形。

第一种是平稳换人，也是目前可能性最大的一种。董事会罢免老黄或者老黄自己辞任，Kress临时接手，之后敲定正式CEO，老黄则保留股东身份，甚至可能留个顾问的头衔。公司的日常运营不会停摆，产品该出货出货，订单该交付交付，要花时间的是组织架构的重建。

第二种是老黄留任，但被关进笼子里——董事会保留他的CEO职位，同时设董事长、设COO、强制做接班计划。这种情形在第二段婚外情曝光前还有可能，现在可能性很低。

第三种是事件升级，比如违反管制。那将面临罚款、长期的合规监管，这才是最关键的。但是，目前没有证据指向这一步。

不管哪一种可能性，受影响的是人、是合规，没有一种是AI芯片的需求没了。

04 对产业链的影响

带着上面的这个结论，我们往产业链上看。

先看下游客户。微软、Meta、谷歌、亚马逊这些云大厂买英伟达的GPU，是因为自己的AI资本开支要落地，是因为英伟达的产品跟CUDA生态目前没有对手，不是因为老黄这个人。

换不换CEO，客户该下的订单一张都不会少。

真正会受影响的是另一类客户，也就是靠英伟达输血的新兴云厂商跟AI创业公司。

老黄这几年有一套独特的打法，英伟达投资客户，客户再拿钱买英伟达的芯片，这套打法高度依赖老黄个人的胆量跟信用。换一个经理人来当CEO，尤其是过渡期，这类投资一定会变保守，那些自己不赚钱、靠英伟达支持才拿得到融资的二线算力公司，才是产业链上最该担心的一环。

再看竞争对手，短期资金已经用脚投票。

周一英伟达大跌的同时费城半导体指数反而大涨近7%，AMD单日涨了13%，博通涨了8%以上。

笔者认为真正受益的是博通跟谷歌。但长期来说整个市场格局没有太大改变，只能是短期消息面影响，因为英伟达管理层动荡的此时，正是云大厂加码自研ASIC的窗口期，博通是ASIC的最大赢家，谷歌的TPU是目前唯一能在系统层面跟英伟达叫板的方案。

最后看上游供应链，也就是台积电、SK海力士、美光，以及鸿海、广达这些整机厂。

上游的订单取决于英伟达的出货量，而出货量取决于下游的资本开支，这条链上没有一个环节是由英伟达的CEO是谁决定的。

就算英伟达换上一个完全支持美国本土制造的CEO，英伟达的芯片还是只有台积电能代工，英特尔没有产能，也没有大幅度增加资本开支，制程水平也落后一代，英伟达想给它也给不了，这是产业事实层面。

但以消息面来解读的话，必然是有利英特尔，不利台积电，但这也只能是消息层面，影响不了真实的产业面。

这两天知名分析师郭明錤也在谈论黄仁勋至于英伟达的价值。

郭明錤的意思是，老黄不一定是英伟达内部最懂HVDC、CPO、CCL这些新技术的人，老黄的价值是同时考虑策略、组织、技术跟供应链，然后迅速拍板。

这跟我的文章前面讲的因果关系是一致的，老黄是拍板的人，不是发明技术的人。

郭明錤给的建议是阶段性降低英伟达对老黄的决策依赖，同时布局接班计划，这是一个慢慢过渡的思路。但实际上，英伟达的章程是董事过半数就能直接换人，董事会当晚就在开会决定去留，没那么拖沓。

郭明錤还认为，老黄的激进决策带动了供应链的规格升级，再带动整个AI产业的技术升级。按他的逻辑，老黄一走，产业的技术升级就要慢下来，这其实是不准确的。

站到整个AI系统的高度往下看，英伟达自己也是被推着走的那一个——推着英伟达走的是需求，是物理极限。

所以这压跟不是老黄激进，老黄是创始人在位30多年，在此之前他一点也不激进，十多年年的显卡，它永远都是用落后一代制程，还多次选择碰便宜的三星代工，AMD在这之前比他激进多了。

AI硬件的技术迭代，是被需求推着走的。很多看起来很激进的新技术之所以被采用，是因为需求已经超过了旧技术的物理限制，所以不得不换，并不是因为老黄这个人激进。

比如HVDC，单个机柜的功率从过去的几十kW，走到现在的一百多kW，再往后是几百kW甚至1MW，原来的低压供电光是铜排就塞不下了，所以只能改成800V的高压直流。

再说CPO。传输速率一代一代往上翻，铜线能跑的距离就越来越短，可插拔光模块的功耗也越来越高，走到某一个点铜跟可插拔都撑不住，那就只能把光引擎搬到芯片旁边。

这些升级没有一项是谁拍脑袋决定的，全是物理极限逼出来的。只要下游对算力的需求还在涨，这些技术就一定要上，换谁来当英伟达的CEO都得做，而且AMD、博通、谷歌的TPU走的都是这些技术路线，它们可没有老黄。

历史也是一个验证。Blackwell的延迟、机柜的过热问题、CPO导入时点的一再后移，都说明真正决定节奏的是物理跟供应链的成熟度，老黄再激进也快不了。

05 全局观

最后简单总结一下。

事件本身，核心只有两个问题，有没有违法，有没有违反公司规则。前者决定事情会不会升级，后者决定老黄的去留。

在这件事情上，一家伟大的公司，创始人离开是早晚的事，英特尔、苹果、微软、台积电都是在创始人离开之后走到了更高的地方，老黄只是用了一种没人想到的方式离开。

下游客户方面，云大厂的订单一张都不会少，要担心的是靠英伟达输血的二线算力公司。

技术方面，AI产业的技术迭代是被需求推着走的，是需求超过了物理限制才逼出新技术，不会因为一家公司换了CEO就停下来。

所以，没有老黄的英伟达不会有什么大的影响，只是大家不习惯而已。

来源：腾讯科技

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领
* 体验资格每人限申领一次
VIP会员·7天畅读卡 | 大师课·入门串讲
扫码
免费领取

限免

540 收藏

分享到:

写评论

请发表您的评论

表情 图片

发布评论

14 条评论

见闻用户
2小时前 中国重庆

历史总是在螺旋中上升，政治负责螺旋，科技负责上升。老黄不过是穿皮夹克的沈一石罢了！

0 回复

Bubu啊Bubu
2小时前 中国辽宁省

為什麼這麼低素質的文章也能上架?

1 回复

MobileUser0530
2小时前 中国广东省

apple 董事会：我当年也是这么想的 （

0 回复

见闻用户
3小时前 中国上海

正常人只能有效管理3-5人，但黄仁勋直接管三十多人

0 回复

> **Bubu啊Bubu** 2小时前 中国辽宁省
> 回复 **见闻用户：**
> 是60個
> 0 回复

Wayne
3小时前 中国澳门

可能发展更好

0 回复

> **Bubu啊Bubu** 2小时前 中国辽宁省
> 回复 **Wayne：**

不可能，有沒有想過老黃出來另立門戶，搶nvda的生意?

0 回复

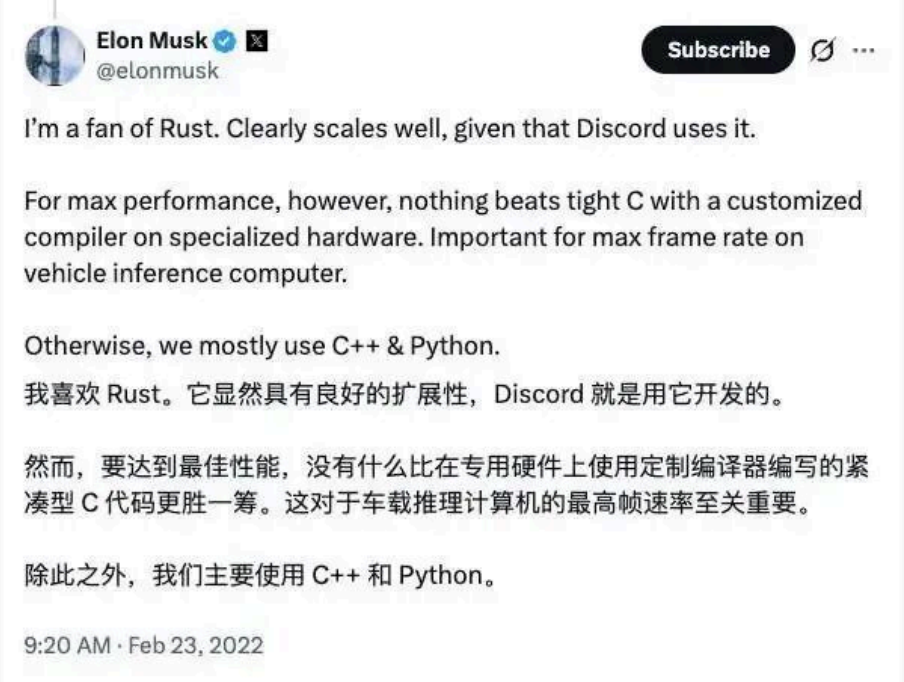
摘要：

Rust 移除垃圾回收、发明所有权模型，是“革命中的革命”。

“真男人自己管理内存。”

11 月 10 日，面对网友“为什么用 C/C++ 重写，之前用的是 Rust 吗”的追问，Elon Musk 用这句话回应了 xAI 的技术选择。

四年前，他还公开表示自己是 Rust 的拥趸，认为 Discord 的实践证明了它的扩展能力。不过，他当时也保留了一个判断：在专用硬件上追求极致性能，精炼的 C 代码配合定制编译器，依然无可匹敌。



如今，随着他宣布弃用 JAX/SGLang、用 C/C++ 从零重写强化学习与推理技术栈，这场关于性能、内存管理和语言选择的争论，又被推到了台前。



在 DTrace 联合作者、Oxide 联合创始人 Bryan Cantrill 看来，Rust 的价值，恰恰要放回过去几十年积累的工程负担中理解。许多语言为了让开发者更轻松，把复杂性藏进运行时：写代码时少操一点心，部署之后却要为垃圾回收、运行时错误和难以预测的性能付出代价。当这些软件成为下一层系统的地基，过去的便利就可能变成后来者无法绕开的技术债。Rust 受到追捧，一个重要原因就是它尝试扭转这笔交易：要求开发者提前想清楚所有权和内存关系，用开发阶段的约束，换取运行时的安全与可靠。

到了 AI 编程时代，这种约束反而更有价值。LLM 可以反复修改代码，编译器则不断指出问题，Rust 陡峭的学习曲线由此显露出另一面。不过 Cantrill 的警告也正在这里：Rust 可以拦住一部分错误，却拦不住 LLM 源源不断地制造复杂性。一个系统可以通过编译、符合内存安全要求，同时充满多余的模块、层层叠加的抽象，以及永远做不完的“改进”。

人会因为脑子装不下整个系统，设法让它更简单；LLM 却没有同样的压力。你让它精简，它可能再派一万个 Agent 来“优化”。Steve Yegge 的软件工厂 Gas Town 就陷入了这样的循环：原本要替人开发软件，最后却只顾着开发自己。按照 Yegge 的自述，Opus 4.7 总觉得“再做两件事就好”，不断修改 Gas Town 本身，始终无法进入真正干活的状态。工厂一直在扩建，却迟迟不肯开工；代码越堆越多，Token 越烧越贵，最终把自己拖垮了。

在 Cantrill 看来，这暴露了 AI 编程更深的风险：模型可以不停写、不停改，甚至把“简化系统”也变成一轮新的扩建。Rust 能检查内存使用是否安全，却无法替人判断，这座工厂究竟还缺一块砖，还是早就该停止施工。

本文基于该演讲视频整理，经 InfoQ 编辑。

每一分开发便利，都是运维者买单

先给大家一点背景，我在 Sun Microsystems——一家现在已经不存在的计算机公司干了 14 年，之后我去了一家云计算先驱公司 Joyent，它在 2016 年被三星收购。然后我们在 2019 年创办了 Oxide。这就是我整个职业生涯的跨度。

我入行那会儿，用事后诸葛亮的眼光看，几乎算是史前时代。那时候大家都觉得面向对象就是未来，但语言对它的支持糟糕透顶。那是 Java 出现之前，只有 C++。如果你觉得现在的 C++ 很糟，那你真该看看九十年代的 C++，更是烂到令人发指。

我很早就认为：编程语言这东西，给不了我太多东西。

我职业生涯是从 C 起步的，1996 年那会儿没人觉得 C 是未来，但我想写的操作系统内核软件，毫无疑问只有 C 可用。但软件世界的其他地方不是这么想的，诉求都集中在语言上：怎么让软件更容易写？怎么让更多人能写软件？

这些诉求听起来都很合理，我们确实希望软件更平民化：一门心思优化开发体验，降低了认知门槛，让开发者不必懂那么多。从汇编和 C 那个年代走出来，这听着很值得称道。可这些东西全都是一笔隐性交易：我们让开发者更轻松了，却让运维者难受得多。这个张力，从我九十年代写 C++ 的时候就埋下了，在今天的 LLM 时代又以一种全新的方式冒了出来。

垃圾回收（Garbage Colletction）就是绝佳例子。垃圾回收让开发者省事，可一旦真要运维这套软件，麻烦就来了，它会伴随软件的整个生命周期。我一个从显式内存分配年代长大的人，都觉得这事有点荒唐：我们居然要把“什么时候要内存、什么时候用完”这种负担从工程师身上卸下来。拜托，这又不是分裂原子。我想要点内存，用完了，还给你，这不难吧？可不行，太“难”了。

这对某些软件确实很棒，但对基础设施类软件，那些要承载其他软件、要扛负载的软件，就是跟你对着干。我认识的那些在垃圾回收语言里写数据库的人，首先，我为你感到难过，你花了大量时间去“讨好”垃圾回收器：有一块自认为没人引用的垃圾，拼命希望垃圾回收能发现它，不然垃圾回收就会 stop the world，给你带来延迟。更糟的是，你以为那是垃圾，其实不是，你手里还攥着引用。写过 JavaScript 的都懂，用闭包太容易创建出你看不见的对象引用了。然后你骂垃圾回收不停地跑，其实它根本没在回收垃圾，它现在是个堆扫描器，纯属南辕北辙。

动态类型也一样。不用预先定义类型、运行时再确定，对开发来说很爽。如果你从静态类型语言转到 JavaScript，那感觉就像进了裸体营地。你从小穿惯了衣服，突然大家都把衣服脱了，一开始是有点刺激，但人穿衣服是有原因的。动态类型也许让某些软件开发更容易，但现在你在生产环境遇到的问题，本该是编译错误，比如本来只是个手误把属性拼错了，现在却要在运行时爆出来，这就是为什么没人该用 Node.js 写基础设施软件。

还有 M:N 线程，我希望大家都不认识这东西。它号称“你能创建几百万个线程”，你真去创建几百万个，一切都卡死了，又是那种为了迁就开发者、却让系统更难理解的东西。

异常也是。在代码深处动态抛异常，感觉很好，不用担心错误处理了。可一旦真出了错，异常就像草丛里的蛇，很难正确处理。尤其 JavaScript 里，如果异常是因为属性拼错导致的类型错误，你根本不知道该怎么处理。

ORM（对象关系映射），“我们想让访问数据库超级简单”，对开发者很棒，开发速度飞快，甚至都不用知道自己是在跟数据库打交道。可真正在生产里处理这摊子的人心想：我就知道，他们不知道自己面对的是数据库。我这辈子花了太多时间在生产环境里调试别人的代码，总觉得这世界有一种因果失衡：一定有人制造的 bug 远多于他们调试的 bug。等你遇到这些人，很难开

口说“嘿，我读过你很多代码，顺便问一句，你知道那些东西全都是数据库查询吗？”这也情有可原，因为抽象就是不让你知道，好让你开发得更快，但代价是引入了所有这些复杂性。

RPC（远程过程调用），几乎从定义上就是个坏主意：你有个远程的函数调用，然后说“别担心你要走网络，就当调个函数”，这让你一不小心就造出一个病态系统。

抽象是软件的全部，而坏抽象会爆炸

这些都是抽象稍微出了问题的例子，所以让咱们聊聊抽象，因为抽象正是软件的全部。我们看不见它、闻不到它，却对什么是干净的抽象有很强的审美。一个好抽象能让我们造出非凡的东西，紧凑到人们甚至不知道自己被抽象掉了。这跟 ORM 那种“不知道”不是一回事：好抽象是无缝到根本看不见，不会在生产环境里引发涌现行为；坏抽象则会泄漏、会爆炸，产出的系统极其脆弱。

当你在开发阶段为开发者过度优化，想把开发者从现实和后果中屏蔽出去时，几乎不可避免地会造出一个泄漏的抽象，而收拾它的人是运维。

垃圾回收就是最好的例子。它本来不该泄漏，前提是我们真的永远不用操心它。但每个 Java 性能问题最后都是“你得换个垃圾回收器”，或者“换个算法”。我们简直是在搞降神会，从替代垃圾回收器的超自然世界里召唤灵魂，这就是恐怖地泄漏的抽象。它对很多系统没问题，但当我们试图构建基础设施，那种我们在其上构建更多软件的软件，它就会坏得很惨。

坏了，它们就引入复杂性。复杂性和抽象是软件工程的一对偶极子。好的抽象把我们蒸馏到问题剩下的本质复杂性上，坏的抽象诱发偶然复杂性。真正的问题在于，复杂性不只是累积，它还会爆炸：坏抽象诱发的偶然复杂性会变成下一层必须处理的本质复杂性。你的坏抽象创造了一个糟糕宇宙的新规则，而我们发现自己就困在里面。

举个非常具体的例子：基本输入输出系统（BIOS），它是系统里非常底层的固件，后来演变成 UEFI。x86 或者 ARM 系统启动时，就会执行这段固件。这挺让人失望，因为 ARM 本来根本不需要模拟这套行为。如果你在实现一项技术，想取代一个你觉得比你走得更远的技术，千万不要假设围绕它运转的复杂性就是本质复杂性。ARM 那帮人觉得，我们 ARM 也必须有 UEFI BIOS，这玩意儿是本质复杂性。才不是，它是偶然复杂性，你其实根本不需要它。但当每个人都用的时候，你就会觉得它不可协商。我们日常要处理的大量复杂性都像是不容置疑的本质复杂性，但其中很多其实是过去选择造成的偶然复杂性。

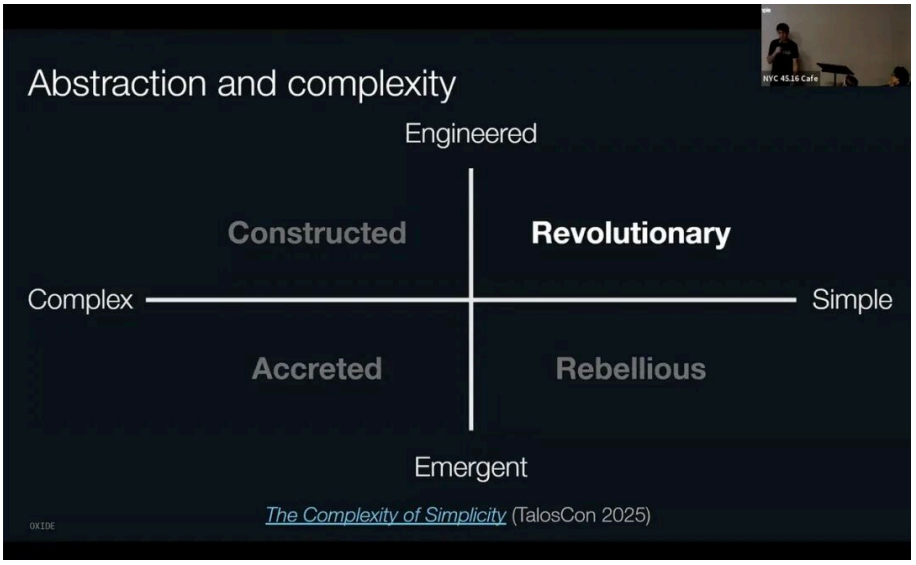
大概一年前我把这种分类做成了一场演讲。我画了两个轴：一个从复杂到简单，从“搞得一团糟”到“优雅”；另一个从涌现到工程化，衡量我们对决策有多刻意、多深思熟虑，而不是多被动、多应激。这样就得到四种系统。

左上角是工程化的、复杂的系统，我们叫它“构造型（Constructed）”。Fred Brooks 那本 60 年代写的《人月神话》到今天仍然切中要害，它描述的就是一个超大型构造型系统的风险：60 年代 IBM 那个 360 系统，规模巨大，很多人同时在上面工作。

它们会变得非常复杂，然后激发出“叛逆型（Rebellious）”系统（译者注：位于第四象限）：我们真的需要做所有这些事吗？要不全扔了？这些复杂性解决的是我们根本没有的问题，“来加入我的叛乱吧”，这其实就是私募股权那种“通过摧毁来创造价值”的幻想。大多数时候他们只是毁灭，但当这类系统真的奏效时，确实丢弃了一堆你并不需要的东西。Multics 到 Unix 就是典型。Unix 这个名字本身就是对 Multics 被阉割的双关，一个刻意叛逆的系统：我们要靠做得少得多来创造点什么。叛逆型系统会在“删得太多”的时候失手：它说“这个其实承重”，而那个被删掉的东西恰恰让系统无法交付了，好在大多数时候被丢弃的都是偶然复杂性。

然后系统常常会落到第三个象限：“累积型（Accreted）”系统。它是涌现出来的，不是设计出来的，但同样复杂。很多系统一开始是构造型或叛逆型，最后都变成累积型。UEFI BIOS 就是百分之百的累积型系统：高度涌现、非常复杂，你极难把本质复杂性从偶然复杂性里择出来，这就是技术债。

有些系统一出生就是累积型的，连推广它的人都说“这很遗憾，但它确实解决了问题”。累积型系统作为地基非常糟糕：里面太多偶然复杂性搅在本质复杂性里，而这份偶然复杂性会变成架在它之上的东西的本质复杂性。不过，被累积型系统压得够久、够重之后，灵感可能会降临：我要做点彻底激进的事，靠创造来创造价值，而不是靠摧毁。



你会拉上一小撮人，给他们专注的时间，去做一件有点野的事。我们要改变抽象，而不只是减少抽象。几乎所有编程语言都落在这个范畴里，因为发明一门语言，几乎就是在发明一种新的抽象。否则你干嘛不直接用 Lua 嵌进去？如果你被系统的累积压得喘不过气，可以去这些“革命性（Revolutionary）”系统里找灵感。

当年的革命，怎么成了今天的技术债？

但这里有个悖论：你正在对付的那个累积型系统，很可能不久前自己就是个革命性系统。它们起步时是革命性的，最后却变成了当初想要颠覆的那个东西。

Java 就是个绝佳例子。去翻翻 1995 年 Gosling 那份白皮书，那时它还叫 Oak，你能体会到 Java 当年那场真正的革命。Java 是对 C++ 的革命，而 C++ 是一团乱麻。C++ 是“出生即累积”的典型：当创造者觉得自己得为这个创造向全世界道歉时，你就知道它是累积型出生的。

Java 是一场革命，但没花太久就变了味。

如果你现在整天泡在 J2EE 系统里，绝不会说“这些抽象多优雅”，只会说这就是一团彻底的乱麻。2005 年的 Ruby on Rails 就是对 Java 复杂性的反作用。Python 也如此：它最初是作为教学语言设计的，有很多好东西，也有太多复杂性。今天你不会把 Python 称作革命性系统，尽管它起步时是。

为什么这些系统最终都走向累积？因为累积成了把这些东西当作生产基础设施来运行的必然属性。二十年前那可能是个好主意，但现在我正试着拿它当地基往上盖东西。

Java 优化了垃圾回收，它是为开发阶段的开发者优化的，我祝 Cassandra 的开发者永远不用操心内存。可我现在要在生产环境里对付 Cassandra，做垃圾回收的降神会，想压低延迟，一边

还盘算着往右上角那片革命性系统的象限看。

我不反对垃圾回收本身，我反对的是，把它当作基础设施部署到生产环境之前，对它会变成什么样一无所知。结果就是，很多革命性系统起步时是革命性的，却实际上变成了一场奔向累积的加速跑。

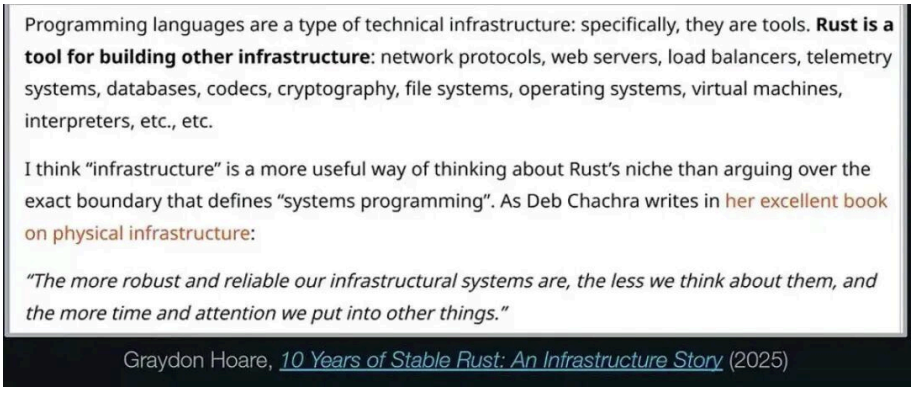
幸运的是，今天你在这里，是因为有一场更深的革命正在酝酿。对任何系统，回到原始文档总是有帮助的。创造者当初到底想做什么？你能学到这个系统为自己做决策时用的那套标准，这非常重要。

就 JavaScript 而言，你根本找不到 Brendan Eich 关于 JavaScript 的基础性文档，这本身就说明了一些问题。JavaScript 某种程度上是“出生即累积”的，他们连名字都没保住，本来叫 LiveScript，后来改成了 JavaScript。如果你曾经是个编程语言新手，问“抱歉我没跟上，Java 和 JavaScript 有关系吗？”没有，它们连表亲都算不上，它们是营销上的表亲。所以回到原始文档很重要，它能告诉你决策为什么这么做。

Rust 真的非常迷人，原因有很多很多。

Rust 的发明者 Graydon Hoare，我和他的渊源说来有点奇怪。我在 Sun 做过一个叫 DTrace 的东西，能在生产环境里观察系统，后来被几个开源项目“致敬”了，其中一个叫 SystemTap。那个项目里只有一个人让我觉得“这人真不错”，而当我发现他离开项目时，我内心阴暗地感到一阵狂喜，因为那意味着我不用再跟那个人打交道了。那个人就是 Graydon，他离开是因为想去做一门新的编程语言。

那是 2004 到 2006 年间的事。我当时想，这太荒谬了。新编程语言已经被过度优化成“让开发者爽”，而对我这种需要造出生产环境可依赖的基础设施的人来说，它们什么也没给我。我那时在用 C，C99 就是我能等到的最大变化，我几乎认命了。



> 我认为，与其争论“系统编程”的确切边界，通过“基础设施”来理解 Rust 的定位更有帮助。正如 Deb Chachra 在她那本关于实体基础设施的优秀著作中所写的：“我们的基础设施系统越稳健、越可靠，我们就越少需要考虑它们，也就越能把时间和注意力投入其他事情。”
>
> ——Graydon Hoare，《Rust 稳定版十周年：一个基础设施的故事》（2025）

但幸运的是，Graydon 真的去把 Rust 造了出来。他后来回顾 Rust 稳定十周年时写了一段话：“编程语言是一种技术基础设施，具体来说，它们是工具。Rust 是构建其他基础设施的工具：网络协议、Web 服务器、负载均衡器、遥测系统、数据库、编解码器、密码学、文件系统、操作系统、虚拟机、解释器等等。”我是个系统人，看到“等等”就想，还有别的吗？这不就是全部软件了吗？

用“基础设施”来理解 Rust 的生态位，比争论“系统编程”的精确边界有用得多。就像 Deb Chachra 那本讲物理基础设施的书里写的：我们的基础设施越鲁棒可靠，我们就越不会去想它，从而把时间精力放到别处。你不需要惦记的基础设施，才是真正好用的抽象。

不会读 Rust，还嫌 Rust 代码丑?

这一点很关键，因为 Rust 做过一些非常了不起的事。如果看那些“为开发者服务”的特性，Rust 至少有两个：绿色线程和垃圾回收。当年我得知这两点时，感觉就像收到了来自那座偏远岛屿的消息，而那座岛正在烧成灰。我当时面无表情地想，好吧，当然。但接着疯狂的事发生了：Rust 把绿色线程删了，把垃圾回收也删了。

移除垃圾回收在我看来是一个极其有趣的时刻，因为它实际上就是所有权模型的发明。我知道这不是 Rust 独有的，我一说独有，就会有人跳出来说 Simula 在 1967 年就干过、Clean 也干过，好吧，我向所有 Simula 和 Clean 程序员道歉。但它绝对不在主流里，这几乎是“革命中的革命”。

Rust 是 Graydon 在 Mozilla 开发出来的革命性系统，但它也开始积累偶然复杂度。Graydon 的目标是造一个作为基础设施工具、能导向“移除”的软件。虽然移除不是他亲手做的，但他创造了一种文化，这种文化导致了垃圾回收和绿色线程的移除，并在很大程度上挡住了异常机制。Rust 把“基础设施”当作约束，先说我们要让产物足够鲁棒、能当基石，然后在这个界限内尽量把开发者体验做好。我们不会为了让开发者舒服，就交出一个不鲁棒、不安全、不可靠的产物。

Rust 社区历史上的很多紧张关系，就来自“这条线到底画在哪”。总体上说，Rust 做得极其出色。我不是说写 Rust 不难，我是说那不是优先级。我们实际上是把认知负担移回给开发阶段的开发者，你得真的去想你的内存。这里有一笔大交易：如果你按“谁拥有什么”来组织程序，Rust 就给你安全。我之前几次顺嘴黑垃圾回收时说“人们只是不会 malloc 和 free”，那有点轻佻。实情更复杂：你在管理内存时，是在一门目的论上不安全的语言里工作，不只是 malloc/free，还有内存损坏、缓冲区溢出。Rust 说的是：我要强迫你，程序员，指明谁在什么时候拥有什么，作为回报，我给你安全。

我 2018 年才接触 Rust，正好在 non-lexical lifetimes 之前。那时编译器天天把你按在地上摩擦，它建议你“把这个改成 mut”，你改完它又说：“谁让你这么干的？”是你自己啊，我只想让代码编译通过而已。我犯了很多 C 程序员都犯的错：我没有真正理解那笔交易，没有去想“我得换一种方式组织程序”，而是把我想当然的结构硬塞进 Rust 的所有权模型里。你确实能在 Rust 里写双向链表，但你真的不该写。双向链表从定义上就是“两个东西拥有同一个对象”。有个很棒的博客系列叫《So You Really Want to Write a Doubly Linked List in Rust》，通篇都在劝你别干，但“你非要的话，来吧”。对 C 程序员来说，这相当于一口气抽完一整包烟。

最终我被击败了。我意识到，算了，我不写双向链表了，我按你的方式来，我眼泪汪汪地只想让它编译通过。我拿一个项目试 Rust，本来想先让它跑起来别管性能，结果它跑起来后，比我写的 C 还快。我几乎被自己冒犯了：我花了很多时间写的 C，怎么会比 Rust 慢?

我去问 Steve Klabnik，这里面肯定有什么精妙的编译器技巧吧。我列了一堆猜测，他说都是好主意，但我们一个都没做。真相简单得多：是抽象的力量。

你想在 Rust 里用 B-tree，用 BTreeMap 就行。我们对此太习以为常，以至于忘了这件事有多么了不起。在 C 或 C++ 里用 B-tree，就像带着枪进幼儿园。因为 B-tree 依赖在内存里移动数据，而 C 里没有“谁拥有什么”的检查。你去找找 C 的开源 B-tree 库，很难找到，因为契约太复杂了。但在 Rust 里，正是因为有所有权模型替我们检查这些问题，我们才能放心地使用

BTreeMap。这有点反直觉：一方面它对你要求更多，另一方面它给你的远多于拿走的。这才是 Rust 真正具有革命性的地方。

尤其在 LLM 时代之前接触 Rust 的人，会有那种非凡的感觉：我的 Rust 一编译通过，它就能工作。重构也一样。在很多语言里重构是自担风险，我觉得重构常常得不偿失，因为它引入风险。但在 Rust 里，你重构完只要编译通过，它大概率就能工作，编译通过本身就是一个数据点。

当然，这也意味着学习曲线陡峭。你刚学 Rust 时，人们说“我害怕那个可怕的编译器”，结果你发现它超级友好，终端里有颜色、有 ASCII 艺术精确指出你的错误。这个友好的编译器是在积攒社交资本，准备在你最黑暗的时刻一次性提现到接近零。学习曲线是陡的，但它给了你巨大的回报。

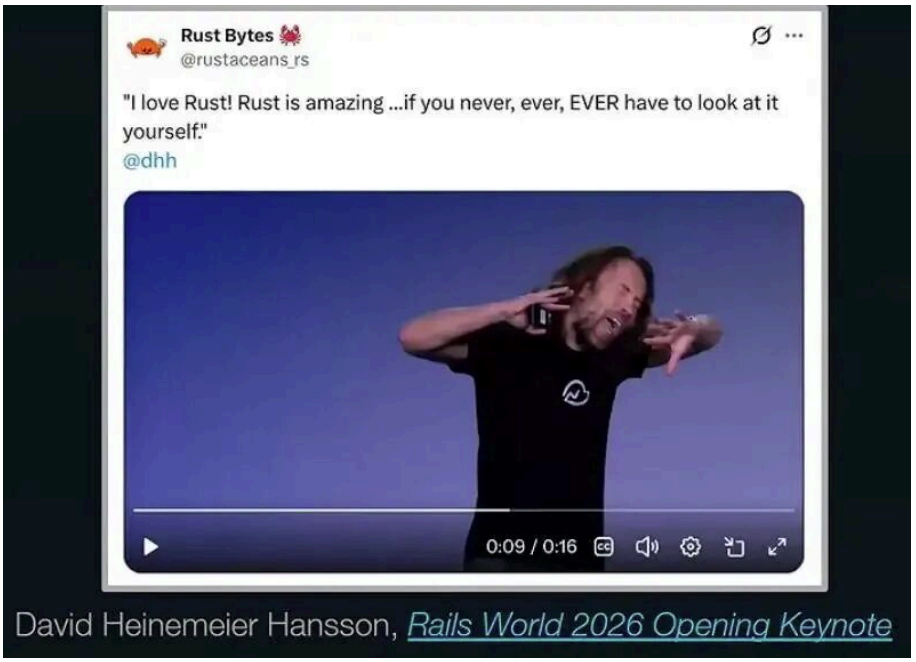
说一个反直觉的结论：Rust 那个被诟病的“学习曲线”，在 LLM 时代反而成了优势。认知负担从生产端转移到开发者这一侧，这其实是件大好事。代码只要编译通过，我们就能对它的正确性说点什么。这种反馈对 LLM 来说简直是完美的。

还有一点：Rust 关于构建鲁棒、安全、快速软件的那些价值观，作为一种基础设施工具，它吸引的是同类的心智，这是 Rust 最重要的事情之一。你走进 Cargo 生态，那里有极其出色的软件。我敢说你们都有过自己的 Serde 时刻，就是那种“我的天”的感觉：一行代码解析 JSON，而且比我以往任何时候解析 JSON 都更鲁棒。我们有很多这样的 crate，正因为我们吸引了那些同类的心智。这个生态给了 LLM 一个非常鲁棒的基础，让它可以放心去 YOLO。你坐在鲁棒的地基上，vibe coding 起来容易得多，而 Rust 的地基极其鲁棒，同时产物本身更好、更快、更安全。

当然，你会想，vibe code 出来的东西有顾虑？没事，它是 Rust 写的，这帮我解决了。不，并没有解决，但它确实很有吸引力。而且我认为很多人会说，“太”有吸引力了，因为接下来就有人登场了。

David Heinemeier Hansson，Rails 的缔造者 DHH，他现在相信一切都应该永远 vibe coding，在 Rails World 上做了一场关于自己怎么用 Rust 的主题演讲。你第一反应是：太好了，DHH 在用 Rust，我们为什么不张开双臂欢迎他？但你最好先把那场演讲看完再说。

实际上，他一边用 Rust 做所有事情，一边在同一时刻贬低 Rust，而且贬低的是 Rust 的美学。美感这东西见仁见智，但有些“见仁”的人，事实证明就是个蠢货。DHH 觉得代码很丑。我特别“欣赏”这一点：这个人读不懂 Rust，还以读不懂 Rust 为荣，只靠 vibe code，然后反过来抱怨代码的质量。你想欣赏一门语言的诗，你总得会说这门语言吧。



他说“我用 Rust 是因为性能惊人”。对，但为什么性能惊人？性能惊人是因为你用的是 BTreeMap 而不是 AVL 树。为什么你能有这个东西？因为所有权模型。你之所以能用 Rust 编程，是因为很多人多年来在这门语言、环境和生态里付出了极其艰苦的努力。你从中受益，同时又在贬低它，这是相当冒犯的。如果你是 Rust 社区的新人，请不要这么做。

AI 越勤快，系统越复杂

这让我想到 LLM 的一堆问题里的一个：LLM 缺少人类最重要的特征之一，它们不懒。Larry Wall 说懒惰是程序员的三大美德之一，另外两个是傲慢（hubris）和鲁莽（impertinence）。我们给操作系统起名叫 Hubris，也许确实是因为我们自己的傲慢，但也确实是在致敬 Larry Wall 说的那个美德。

Larry Wall 是 Perl 的作者。他说的懒惰指的是：把系统装进我们脑子里、受限于自身认知边界的需求，会驱使我们走向优雅而简单的抽象。我们当年在一个不断堆积的系统里发起革命，部分原因就是我们理解不了那么多复杂性。在一个堆积出来的系统里是没有第一性原理的，而第一性原理往往就来自懒惰。我敢说在座不止我一个人对三角学只记得一件事：$\cos^2 x + \sin^2 x = 1$，剩下的考试时现推。只要掌握第一性原理，三角函数照样能考得不错。

而 LLM 完全没有这个。LLM 热爱工作，所以它们强烈偏向复杂，不只是复杂问题，而是复杂本身，它们会乐呵呵地为你制造复杂性。这很糟糕，我想让它懒一点。当然你可以像对付经典 LLM 那样抽它一巴掌，让它“懒一点”，它会道歉，然后非常努力地去变懒。

我的意思是，去思考 Rich Hickey 那套著名的“吊床驱动开发”（hammock driven development），真的去琢磨这个系统。结果它说：明白了，我派一万个 agent 去琢磨这个系统。算了，可能还是别了。

更糟的是我们还有所谓的管理咨询顾问效应。我上学那会儿，管理咨询是数量可怜的一小撮院校本科生的常见去向。你要是 90 年代上了常春藤，有朋友去当管理咨询顾问。你可能会像我一样想：你二十分钟前还在玩酒桶倒立（keg stand），凭什么有人听你的？但这个理念，麦肯锡就是极致代表，是“我们要找最优秀最聪明的人”。对，就是 McNamara 那个贬义上的“最优秀最聪明”：最优秀最聪明的人，却不懂越南其实是一场内战。

他们去给 CEO 和高管层做咨询，最后往往形成一种有毒关系。要么他们自己根本不知道自己在干嘛，一个 22 岁、聪明但什么都不懂的年轻人居然能替你运营业务？要么更常见的是：我其实

就想让你背书我本来就打算做的决定，或者“帮我打开眼界，我们可以把阿片类药物更好地营销给医生，太感谢了麦肯锡”。

你手里那些 LLM，就是你的小麦肯锡咨询顾问。你要是放任它们，它们会和你形成一种共生依赖。它们很难把事情“做完”，就像管理咨询顾问希望周一出现在你面前。麦肯锡出了名地周一到周四驻场，他们想周一再回来。你问“做完了怎么办”，他们说“什么叫做完？永远有更多事可做”。这是实实在在的危险，因为 LLM 明里暗里就是为参与度（engagement）设计的。

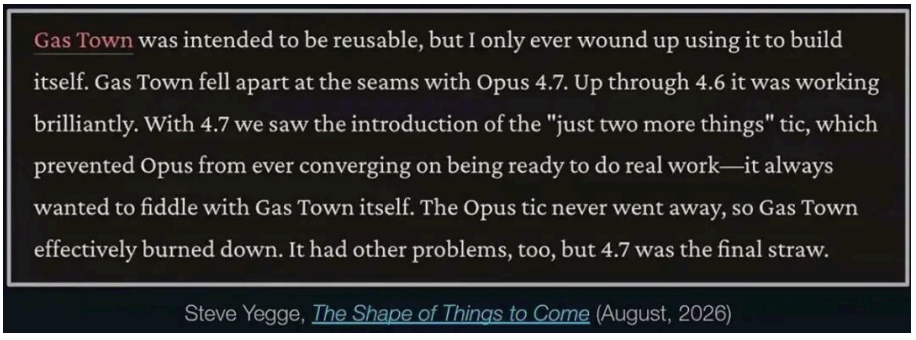
管理咨询顾问还是个厚道的比喻，它也可能是一台意外造出来的老虎机。人们常说“我用 LLM 用得精疲力尽”，那你有没有在赌场待过九个小时？是不是同一种感觉？这些东西为参与度而设计，这本身不算错，我们其实也想要这个。但当一个为参与度设计、长得像管理咨询顾问、又没有人类“懒惰”特质的东西来替你设计软件时，你最好小心点。

因为你可能得到能跑的东西，然后你发现问题，它说“别担心，周一还有 200 个咨询顾问落地，我们全给你重写”。你说这不是我想要的，我想让这东西更紧凑更优雅。“明白，我们派 300 个过去，让它更优雅”。所以你得意识到它可能是台老虎机，安息吧，Gas Town。

为什么 Gas Town 最终会走向失败

Gas Town 是 Yegge 开发的一个软件工厂，风格大概像是理查德·斯卡里遇上《疯狂的麦克斯》。它是用 Go 写的，不是 Rust，但 Yegge 吹嘘过一件事：“我从没看过这些代码，也不打算看。”你听了就想，真的吗？你就让那台麦肯锡老虎机把代码全写了？行吧，那后来呢？

后来就悄悄发生了，Gas Town 烧没了。我觉得这事挺有意思的。第一，它不知道该怎么停下来。你看过 LLM 互相打电话、却不知道怎么挂断吗？就是这个，只不过发生在你的软件上。它永远不收敛，不停给自己找新活儿干。与此同时，旗子倒下、计价器还在跑，这也很贵，而且 Opus 那档模型的消耗始终没停。



> Gas Town 原本被设计成一个可复用的工具，但到头来，我只用它来构建它自己。到了 Opus 4.7，Gas Town 开始全面崩溃。在 4.6 及此前的版本中，它一直运行得非常出色。可到了 4.7，模型开始出现“再做两件事就好”的毛病，导致 Opus 始终无法收敛到可以开始干正事的状态——它总想继续折腾 Gas Town 本身。Opus 的这个毛病始终没有消失，所以 Gas Town 实际上就这么烧毁了。它也存在其他问题，但 4.7 是压垮它的最后一根稻草。
>
> ——Steve Yegge，《未来事物的形状》（2026 年 8 月）

LLM 无可争议地极其强大。我知道确实有人说“我永远不用”，但我们太沉迷于乌托邦或反乌托邦的未来想象，反而忘了：这些东西在理解力上是爆表的。如果你手头有一个代码库，我建议用 LLM 去问关于你自己软件怎么工作的问题，而且这些答案可以随手验证。你不用去担心老虎机效应、麦肯锡效应那一套，就问“这部分是怎么运作的”，它给出的答案出奇地好，还带出处。所以它在理解层面极其强大，生成层面也强大，但你得睁着眼睛用。

我们同事 Rain 有个技巧我特别喜欢：她不让 LLM 替她写注释，注释要自己写。理由是，如果没法为它写出一条注释，说明她根本没搞懂发生了什么。她可以跟 LLM 讨论，搞清楚状况，再自己把注释写出来，用这个动作逼自己真正去看它做的每一件事。

我觉得这是个很有意思的技巧，我们还会发现很多这类“如何真正有效地使用这些工具”的门道。但核心是：你要把它当工具用。任务定义得越清晰，你成功的概率越高；你把口子开得越大、让它钻进复杂地带，它就越疯，Gas Town 被烧掉的概率就越高。

而且我们 Oxide 坚信一件事：LLM 可以让我们变得更严谨。程序证明这事，如果当年你告诉我，编程语言开发会给我这个系统程序员带来什么，我做梦都想不到程序证明会离我这么近。形式化验证？绝对不可能。结果呢，形式化验证现在真的成了实打实的东西，部分原因就是 LLM 能扛下大量重活。但我们不希望 LLM 取代软件工程的核心本质，那个本质就是创造抽象。

学 Rust，代码仍然重要

所以我要说，如果你是 Rust 新手，去学 Rust。哪怕你用 LLM 生成了大量 Rust 代码，你也不希望自己患上习得性无助。你不希望成为那种没法调试自己代码的人，不希望成为那种必须靠 LLM 才能看懂 LLM 写了什么的人。

你应该学 Rust，而且我要告诉你：你能学会 Rust。任何学 Rust 的人都要先跨过一道坎，那种“Rust 高不可攀”的神秘感。但如果你心里想“我不知道我能不能学会 Rust”，你能学会。好消息是，现在有个很棒的新工具能帮你学东西，它叫 LLM。

我们正处在全新的领域。我有三个孩子，看着 19 岁那个上大二的儿子天天用 LLM 学习，特别有意思。他告诉我：我有了一个家教，有了一份学习指南。当然，因为高等教育现在同时经历八场不同的存在主义危机，他所有的考试都改成了课堂手写，还放在法拉第笼里，你没法作弊。但对他来说，把 LLM 当学习辅助工具，效果是惊人的。用 LLM 来学习，多么激进的想法。但它是可行的，完全可以用这些东西来增强我们的理解。

所以我真心劝所有人：请去学 Rust。不要以“我不读 Rust”为荣，如果你真以此为荣，拜托别在 keynote 上讲。如果你非要在 keynote 上讲，也别对着一群用别的语言的人讲，那看起来既刻薄又奇怪。

还有，记住 Rust 的使命，记住 Graydon 的使命。你造的东西，是别的软件要依赖的东西。我相信 Rust 是一场如此重要的革命，我认为我们正处在软件领域的某种文艺复兴。我知道现在感觉不像，感觉我们正在被 LLM 去人性化，但其实没有。因为我们正在生成后代将要依赖的基础设施，代码依然至关重要。

Q & A

观众：你在 Oxide 或者个人使用中，把 LLM 看成是做减法还是做加法？

Bryan：肯定是加法，但它需要纪律才能成为加法。这有点像那句据说是 Henry Ford 说的话：你觉得你能行或者你觉得你不行，两种情况下你大概率都是对的。如果你认为 LLM 是加法，你就会用出加法的效果；如果你认为它是去人性化的，你大概率也会用出去人性化的效果。

LLM 在很多维度上都极其非凡，而我们其实还没真正理解它。我还记得互联网出现之前的日子，在维基百科之前，你只能靠酒吧里的争论。而现在你可以随时问任何问题，得到一个来源可靠的答案，这太惊人了。

我写了 RFD 576，讲的就是 Oxide 内部怎么用 LLM，我特别建议大家去读。让我惊讶且沮丧的是，真正把这方面的思考成文、制度化的公司少得可怜，或者写了但没公开。

有一件事我特别推荐其他公司也做：Oxide 所有公开写作，我们都坚持要“Pangram clean”（Pangram 是一个 LLM 检测器），必须是人写的。原因很简单：别让你的公司或你自己发布 LLM 写的东西，因为大家都看得出来。不只是那些破折号，而是信息密度太低了，你的大脑会直接拉弹射把手，我真的读不下去那个句子。你这样做，是在阻止自己的信息传达出去。别让你的公司这么干，别让你的基金会这么干，说的就是你，Rust Foundation。他们有一篇博客是 LLM 写的，我们点出来了，他们改了，这很好。至于我们怎么用 LLM，我强烈建议每家公司开发自己的评分标准（rubric），这个结构可以拿去用，不是专有的。

观众：你和 Rich Hickey 似乎在很多事上看法一致，比如支持简单性、反对 ORM。但他开发 Lisp，那是一门动态语言，还带 JVM 上的垃圾回收。你怎么解释你们俩在垃圾回收和动态类型上的分歧？

Bryan：可能只是我们写的软件类型不同。我只通过演讲了解他，他的演讲非常精彩，但我不总是同意他说的每一句话，因为我们对系统的视角不一样，我写的软件在栈里更深的位置。他总不会用 Lisp 写操作系统或固件吧？

另外我想说，我在垃圾回收这个问题上可能有点说过头了。我想向那些被我不小心冒犯到的、支持垃圾回收的人道个歉，垃圾回收有它存在的权利，只是它不适合我想写的那类软件。

观众：我们并不是在写持久化 map、原子 ref 和全套 Clojure，那样的开发者很少，但理念似乎是相通的。最近有篇新博客叫《Simple Made Inevitable》，它基本上是在说，有了 LLM，Rust 获得了一些好处，因为那道门槛被降低了，对 Clojure 也是一样。我想听听你对这个的看法，还有你觉得其他编程语言会不会也出现类似变化。

Bryan：我确实认为，这就是我们身处的有趣时代。有些系统一直被贬低，因为它们小，或者像 Clojure 那样，又小又有点怪。但 LLM 让人惊叹的一点是，一个社区的规模，已经不再是它软件发展的障碍了。如果开始出现一些系统开发，是带着“如何正确地与 LLM 交互”来设计的，我一点都不会惊讶。

但我也觉得这很危险，眼下这件事直接变成一团乱麻、把一切拖垮的概率相当高。我们正处在一个你以后会反复回味、甚至可能在法庭证词里回忆的时期。未来会有年轻人问你“那是什么感觉”，你会说：天啊，太诡异了。所以我不会因为一个社区小就贬低它，因为你现在有能力去做远超你规模的事情，只要你驾驭得当。

观众：那你怎么看现在软件工程里的“手艺”？我觉得在一家公司里，很难有理由花大量时间去所谓“传统编码”。我觉得享受写代码这个动作本身、并且保住自己的技能，是很重要的。

Bryan：百分之百同意。我会去别的机械化革命里找安慰，你看木工行业里电动工具是怎么被采用、又是怎么被拒绝的。我们不想丢掉自己的手艺，手艺非常重要。所以暗示“代码没那么重要”、暗示“我们不该盯着代码看”，其实就是在贬低这门手艺，这是个真问题。

我们的招聘流程非常“重写作”，我们会问候选人“你什么时候最不开心”之类的问题。我也知道现在有很多公司和管理极其糟糕的组织，你觉得自己“没法花时间把事情做对”，这真的很不幸，你值得拥有一个和你抱有同样信念的组织。我相信会再来一次文艺复兴，手艺会被重新表达、被重新背书，只是当下确实是个艰难的时期。

观众：你觉得 Rust 里还有哪些特性算是“错失的特性”，或者说被“回退”了？我个人会提名 async。

Bryan：我并不是很喜欢 async，原因有很多。async 取消安全性，集中体现了我在抽象层面反感 async 的那些问题。那些论证“我们不必担心 async 取消安全性”的说法，和 C 程序员说服自己“我们不必担心内存安全”的论证，惊人地相似。它基本上是无法推理的。

我们还遇到过另一个问题，我们称之为 futurelock，它同样是一种涌现复杂性。不过我想加一个重要的星号：我们遇到的问题实在太少了，以至于会专门去谈它们。在我们开发过的其他大型分布式系统里，失败简直是环境性的、无处不在的。而在 Rust 里我们完全没有这些。所以 async 取消安全性这些问题确实存在，但整体体验真的非常出色。

互联网根本没有这些反乌托邦的东西。回头看，一堆东西当时就觉得是个好主意，比如社交网络，感觉棒极了，有什么问题呢？然后就是“不不不，把它塞回盒子里去”。

我真的认为，等尘埃落定，LLM 很可能是一个“互联网级”的事件。每个时代的人都觉得自己独一无二，我们从农业社会过渡到工业社会，大概只用了一代半人的时间；美国电气化全国用了大约 15 年。收音机、电视，都带来了一堆问题，所以这个想法本身并不新鲜。

但油门踏板现在确实像是踩到了底板上。虽然我职业生涯里也有过其他类似时刻，但此刻座舱里每一个 dot-com 时代的警示灯都在闪。同时我也得说，我对前方的能见度，比职业生涯中任何时候都低。

观众：你是怎么在你的构建系统和流程里培养 LLM 纪律的？

Bryan：这个问题我会直接把你引向 RFD 576。它给出了一个非常好的评分标准，说明了我们正在用的东西。

来源：InfoQ

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次 VIP会员·7天畅读卡 | 大师课·入门串讲 扫码 免费领取

限免

111 收藏 分享到:

写评论

请发表您的评论

表情 图片 发布评论

1 条评论



比宇宙更遥远的那棵树

4小时前　中国四川省

太能写了，这么长

0　回复

摘要：

骆晓东表示，作为与CPU直连的核心内存，RDIMM具有低延迟、产业生态成熟、单位容量成本较优等优势，仍是服务器内存市场的重要组成部分。随着服务器内存容量持续提升，市场需求正逐步向中高容量产品集中。

11月12日，第二十三届中国国际半导体博览会（IC China 2026）开幕。长鑫科技副总裁骆晓东在大会演讲中表示，受全球算力需求激增推动，算力超级周期正在进入一个全新的“分层时代”。不同细分市场的增长节奏、不同AI应用对内存的需求正在出现明显分化，存储产业竞争也由单一性能指标比拼，转向全场景产品布局能力的竞争。

骆晓东指出，这种分层首先体现在市场需求上。目前，服务器市场需求占比已超过50%，需求主要来自AI大模型、智算中心等高价值场景；汽车市场预计到2030年复合增速约30%；手机、PC等消费电子市场则随着AI功能渗透加快，持续释放新的存储需求，但仍具有明显的周期波动特征。

与此同时，不同AI应用对内存的需求也呈现差异化特征。例如，大模型推理解码阶段对内存带宽要求较高，长文本前缀处理阶段对算力的要求更为突出，而超大参数、多卡训练场景则对内存容量和互联能力提出更高要求。

在此背景下，存储厂商需要针对不同应用场景提供差异化解决方案。此次IC China展会上，长鑫科技以服务器、手机、汽车三大场景为主线，集中展示最新内存产品及技术进展。

在算力需求最为集中的服务器领域，AI应用对内存容量和带宽的要求持续提升，推动服务器内存向高容量、高速率方向演进。

骆晓东表示，智能体（Agentic AI）的兴起进一步推动服务器平台升级。多租户、高并发等应用特征，对CPU性能、内存通道以及系统整体的数据处理能力提出更高要求。

作为与CPU直连的核心内存，RDIMM具有低延迟、产业生态成熟、单位容量成本较优等优势，仍是服务器内存市场的重要组成部分。随着服务器内存容量持续提升，市场需求正逐步向中高容量产品集中。

围绕这一趋势，长鑫科技持续推进DDR5 RDIMM和MRDIMM产品布局。目前，其DDR5颗粒已从24Gb迭代至32Gb；RDIMM模组实现32GB至128GB主流容量段覆盖，速率涵盖5600Mbps至8000Mbps。同时，MRDIMM产品覆盖128GB和192GB容量，最高速率达到12800Mbps，以满足数据中心对大容量、高带宽内存的需求。

在端侧AI领域，智能手机正成为存储技术升级的重要应用场景。本次展会上，长鑫科技展示了全球首发量产的LPDDR6产品。

据介绍，该产品最高传输速率达到12800Mbps，较上一代LPDDR5X带宽提升60%，功耗降低20%；单颗粒容量为16Gb，芯片最高容量为16GB。目前，长鑫LPDDR6已首批搭载于小米18 Fold折叠旗舰手机，实现LPDDR6产品首次商用落地。与此同时，长鑫LPDDR5X也已应用于全球首款AI智能体手机，为端侧AI交互提供存储支持。

随着AI功能向手机等终端加速渗透，设备需要处理的数据量和运行任务复杂度不断提升，对内存带宽、容量及功耗控制提出了更高要求。LPDDR6等新一代产品的商用，也为端侧AI应用进一步

发展提供了硬件基础。

汽车则是长鑫科技重点布局的另一应用领域。随着智能驾驶、智能座舱等功能不断发展，车载系统对内存的需求持续增长。与消费电子产品相比，车规级内存还需要应对宽幅温度变化、振动等复杂工况，对产品可靠性和功能安全提出更高要求。

目前，长鑫科技已实现LPDDR4X、LPDDR5X车载内存规模量产，并持续推进LPDDR6在车载场景的应用创新。长鑫最新车规级LPDDR5X产品已通过AEC-Q100 Grade 2认证，并获得ISO 26262 ASIL D产品功能安全认证。相关管理体系还涵盖ISO 9001、IATF 16949及QC 080000等认证，为车载内存产品的可靠性和安全性提供保障。

据长鑫科技2026年第三季度业绩公告，前三季度长鑫科技研发投入达到110.71亿元，同比增长73.03%。报告期内，公司在新产品研发方面取得进展，基于第六代技术平台开发的DRAM产品已完成工程样片验证。此前，公司已实现LPDDR6全球首发、第五代技术平台量产，4F²架构产品也取得重要进展。

长鑫科技在三季度业绩交流会上表示，目前公司长协相关工作进展顺利，已与多家头部客户协商长期供货协议，以锁定中长期需求。

对于后续市场，长鑫科技表示，受AI应用快速发展带动，四季度服务器客户需求依然强劲，DRAM供不应求的态势预计仍将延续，公司四季度营收有望继续保持增长。展望2027年，在行业需求持续增长的背景下，公司预计营收仍将保持增长。

来源：澎湃新闻

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领 * 体验资格每人限申领一次
VIP会员·7天畅读卡 | 大师课·入门串讲
扫码 免费领取

限免

75 收藏

分享到:

写评论

请发表您的评论

表情 图片

发布评论